Laporan Analisis dan Implementasi Perbandingan Layered vs Non-Layered Web Application Architecture

Disusun untuk memenuhi tugas mata kuliah Pemrograman Web dan Mobile



Disusun oleh:

Abdillah Ridho Alriza	(2504418)
Faiz Muhammad Ghazali	(2504531)
Hafiz Rifai	(2508874)
Irsyad Afif Musyaffa	(2508023)
Muhammad Fadey Rafif	(2504792)

PROGRAM STUDI ILMU KOMPUTER

UNIVERSITAS PENDIDIKAN INDONESIA

2026

DAFTAR ISI

DAFTAR ISI 1
BAB I PENDAHULUAN 2
1.1 Latar Belakang 2
1.2 Tujuan 2
1.3 Ruang Lingkup Fitur 2
BAB II LANDASAN TEORI 3
2.1 Non-Layered (Monolitik Sederhana) 3
2.2 3-Layer Architecture 3
2.3 REST API dan JSON 3
BAB III RANCANGAN DAN IMPLEMENTASI 4
3.1 Lingkungan Pengembangan 4
3.2 Aplikasi Pertama: Non-Layered Architecture 4
3.3 Aplikasi Kedua: 3-Layer Architecture 5
3.4 Rancangan Database 9
BAB IV HASIL IMPLEMENTASI 10
4.1 Pemenuhan Fitur 10
4.2 Tangkapan Layar 11
BAB V ANALISIS PENGUJIAN DAN PERBANDINGAN 14
5.1 Perbandingan Kuantitatif Kode 14
5.2 Perbandingan Kualitatif 14
BAB VI KESIMPULAN DAN SARAN 16
8.1 Kesimpulan 16
8.2 Saran 16
Lampiran 18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Aplikasi web modern umumnya terdiri dari tiga tanggung jawab utama: menampilkan antarmuka kepada pengguna, menjalankan aturan atau proses bisnis, dan mengelola penyimpanan data. Pada aplikasi kecil, ketiga tanggung jawab ini sering ditulis bersama dalam satu file. Pada aplikasi yang lebih besar, tanggung jawab tersebut dipisahkan ke dalam lapisan (layer) yang berdiri sendiri, yang dikenal sebagai 3-Layer Web Application Architecture.

Laporan ini membandingkan dua pendekatan tersebut melalui implementasi nyata, yaitu Non-Layered Architecture dan 3-Layer Architecture, lalu menganalisis perbedaannya dari sisi struktur kode, kemudahan pemeliharaan, pengembangan, pengujian, dan keamanan.

1.2 Tujuan

1. Membangun aplikasi web CRUD dengan pendekatan Non-Layered menggunakan PHP Native dan MySQL.
2. Membangun kembali aplikasi serupa dengan pendekatan 3-Layer (Presentation, Business Logic, Data) yang berkomunikasi melalui REST API dan JSON, tetap menggunakan PHP Native tanpa framework.
3. Membandingkan kedua pendekatan secara objektif berdasarkan kode yang telah dibuat.

1.3 Ruang Lingkup Fitur

Sesuai ketentuan tugas, aplikasi harus memiliki enam fitur: tambah pelanggan, tampil pelanggan, edit pelanggan, hapus pelanggan, pencarian berdasarkan nama atau email, serta validasi kelengkapan data dan keunikan email.

BAB II

LANDASAN TEORI

2.1 Non-Layered (Monolitik Sederhana)

Pada pendekatan ini, kode tampilan (HTML/CSS), proses bisnis (validasi dan aturan), dan akses database (koneksi dan query SQL) berada dalam satu file atau beberapa file tanpa pemisahan tanggung jawab yang tegas. Pendekatan ini cepat dibuat dan cocok untuk aplikasi kecil, tetapi menjadi sulit dirawat ketika aplikasi membesar.

2.2 3-Layer Architecture

Tabel 2.1 Pembagian Tanggung Jawab dan Contoh Teknologi pada 3-Layer Architecture

Layer	Tanggung jawab	Contoh teknologi
Presentation Layer	Menampilkan data dan menerima input pengguna	HTML, CSS, JavaScript
Business Logic Layer	Validasi, aturan bisnis, pengambilan keputusan	PHP
Data Layer	Menyimpan, mengambil, mengubah, dan menghapus data	MySQL, PDO

Keuntungan utama arsitektur ini adalah separation of concerns, maintainability, scalability, security (database tidak diakses langsung oleh pengguna), dan reusability (API backend dapat dipakai oleh web maupun mobile).

2.3 REST API dan JSON

REST API menggunakan metode HTTP untuk merepresentasikan operasi data: GET (membaca), POST (menambah), PUT (mengubah), dan DELETE (menghapus). Data dipertukarkan dalam format JSON sehingga frontend dan backend dapat dikembangkan secara independen.

BAB III

RANCANGAN DAN IMPLEMENTASI

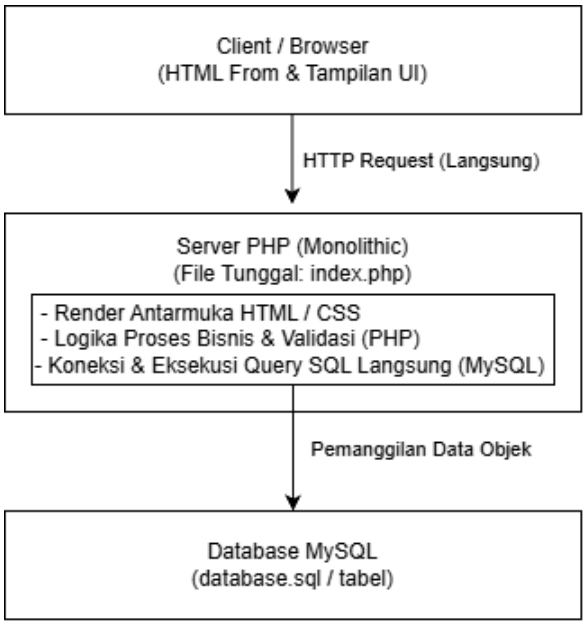
3.1 Lingkungan Pengembangan

Tabel 3.1 Lingkungan Pengembangan Sistem (Development Environment)

Komponen	Keterangan
Bahasa backend	PHP Native (tanpa framework)
Database	MySQL (XAMPP, user root, tanpa password)
Frontend 3-Layer	HTML, CSS, JavaScript (Fetch API)
Server	Apache (XAMPP)

3.2 Aplikasi Pertama: Non-Layered Architecture

Aplikasi pertama dibangun menggunakan pendekatan monolitik sederhana tanpa pemisahan layer secara tegas. Seluruh kode antarmuka HTML, validasi logika PHP, dan query MySQL disatukan di dalam berkas index.php. Alur arsitekturnya digambarkan sebagai berikut:



Gambar 3.1 Arsitektur Aplikasi Non-Layered

Aplikasi terdiri dari satu file index.php (sekitar 190 baris) dan satu file SQL (db_pelanggan.sql). Di dalam file tersebut terdapat tiga bagian yang bercampur:

1. Koneksi database menggunakan mysqli yang ditulis langsung di awal file.
2. Proses bisnis dan query: validasi input (kolom wajib diisi, format email, format telepon 8–15 digit), serta query INSERT, UPDATE, DELETE, dan SELECT dengan prepared statement. Keunikan email dijaga constraint UNIQUE di database; error duplikat (kode 1062) ditangkap dan ditampilkan sebagai "Email sudah terdaftar".
3. Tampilan: HTML dan CSS yang disisipkan bersama blok PHP (<?php if ... ?>) untuk menampilkan form dan tabel.

Alur kerja: browser mengirim form (HTTP POST) atau tautan (HTTP GET) ke index.php; PHP memvalidasi dan menjalankan query; hasilnya langsung dirender menjadi HTML dan dikirim kembali ke browser. Setelah operasi tulis berhasil, halaman di-redirect (pola Post/Redirect/Get) dengan pesan sukses.

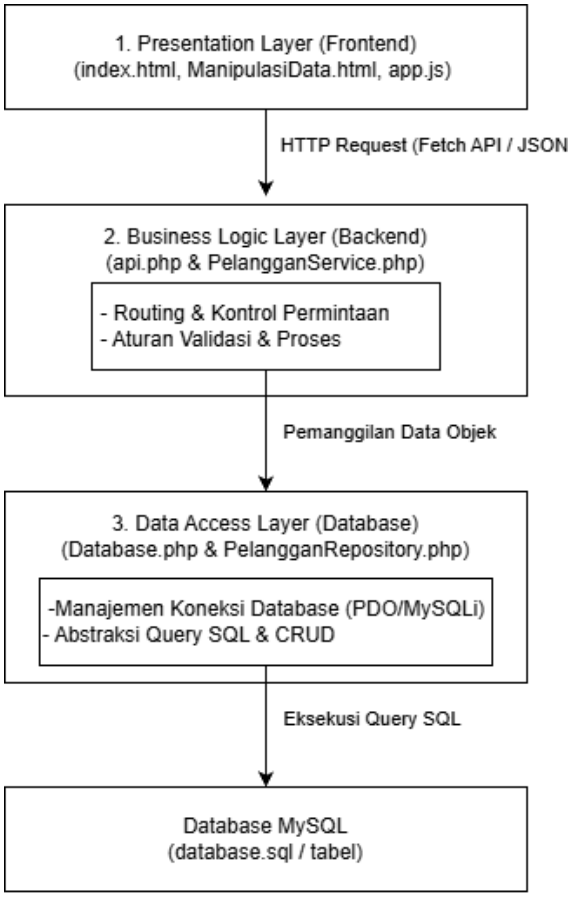
```
Browser  <--HTTP-->  index.php (HTML + validasi + SQL)  <-->  MySQL
```

Struktur berkas

```
1-Layer/
├── index.php          (koneksi + logika + tampilan)
└── db_pelanggan.sql   (skema dan data awal)
```

3.3 Aplikasi Kedua: 3-Layer Architecture

Aplikasi dipisah menjadi folder frontend dan backend dengan pemisahan tanggung jawab (Separation of Concerns) yang jelas. Alur arsitektur dan interaksi antarkomponen aplikasi dapat dilihat pada diagram berikut:



Gambar 3.2 Arsitektur Aplikasi 3-Layer

Penjelasan Diagram Alur:Presentation Layer (Frontend):

- Berkas index.html dan ManipulasiData.html bersama app.js menangani antarmuka pengguna. Saat pengguna bertransaksi, app.js mengumpulkan data dan mengiringnya ke backend via HTTP Request (Fetch API) dengan payload JSON.
- Business Logic Layer (Backend): Berkas api.php menerima permintaan HTTP, menentukan rute, lalu memanggil PelangganService.php untuk memvalidasi input dan menjalankan aturan bisnis.
- Data Access Layer (Data): PelangganService.php memanggil PelangganRepository.php yang terhubung via Database.php untuk mengeksekusi query SQL (SELECT, INSERT, UPDATE, DELETE) ke basis data MySQL.

Aplikasi dipisah menjadi folder `frontend` dan `backend` dengan pembagian tanggung jawab sebagai berikut.

Tabel 3.2 Alur Pemrosesan Permintaan pada Aplikasi 3-Layer Architecture

Komponen	Berkas	Peran
Presentation Layer	index.html, ManipulasiData.html, style.css, js/app.js	Menampilkan halaman, menangkap input, memanggil API lewat Fetch
API / Controller	backend/api.php	Menerima request HTTP, memetakan metode (GET/POST/PUT/DELETE) ke service, mengembalikan JSON dan kode status
Business Logic Layer	backend/services/PelangganService.php	Validasi kelengkapan, format email, keunikan email, pengecekan data ada atau tidak
Data Layer	backend/data/PelangganRepository.php	Seluruh query SQL (CRUD, pencarian, cek email)
Koneksi	backend/config/Database.php	Pembuatan koneksi PDO
Database	databasedummy.sql	Skema tabel pelanggan dan data contoh

Struktur berkas

```
tugas-3layer/
├── frontend/
│   ├── index.html
│   ├── ManipulasiData.html
│   ├── style.css
│   └── js/app.js
├── backend/
│   ├── api.php
```

```
│   ├── config/Database.php
│   ├── services/PelangganService.php
│   └── data/PelangganRepository.php
├── databasedummy.sql
└── README.md
```

Alur permintaan (contoh: tambah pelanggan)

Tabel 3.3 Alur Permintaan

Tahap	Layer	Aktivitas
1	Presentation	Pengguna mengisi form dan menekan tombol; `app.js` mengirim `POST backend/api.php` berisi JSON
2	API	`api.php` membaca body JSON dan memanggil `PelangganService::createCustomer`
3	Business Logic	Service memvalidasi data wajib, format email, dan keunikan email
4	Data	Repository menjalankan `INSERT` dengan prepared statement PDO
5	API	Service mengembalikan array hasil; `api.php` mengirim JSON dengan kode 201 (atau 422 jika validasi gagal)
6	Presentation	`app.js` menampilkan pesan dan memuat ulang tabel

Desain REST API

Tabel 3.4 Desain REST API

Metode	Endpoint	Fungsi	Kode respons
GET	backend/api.php?search=...	Tampil dan cari pelanggan	200
POST	backend/api.php	Tambah pelanggan	201 / 422
PUT	backend/api.php?id=...	Ubah pelanggan	200 / 404 / 422
DELETE	backend/api.php?id=...	Hapus pelanggan	200 / 404
Lainnya	–	Tidak didukung	405

3.4 Rancangan Database

Aplikasi 3-Layer memakai tabel pelanggan pada database db_customer_management:

Tabel 3.5 Atribut Pelanggan

Kolom	Tipe	Keterangan
id	INT AUTO_INCREMENT	Primary key
nama	VARCHAR(100)	Wajib
telepon	VARCHAR(20)	Wajib
email	VARCHAR(100)	Wajib, UNIQUE
alamat	TEXT	Wajib
created_at	TIMESTAMP	Default waktu saat ini

Aplikasi Non-Layered memakai tabel pelanggan pada database db_pelanggan dengan kolom yang sama (id, nama, email UNIQUE, telepon, alamat, created_at).

BAB IV

HASIL IMPLEMENTASI

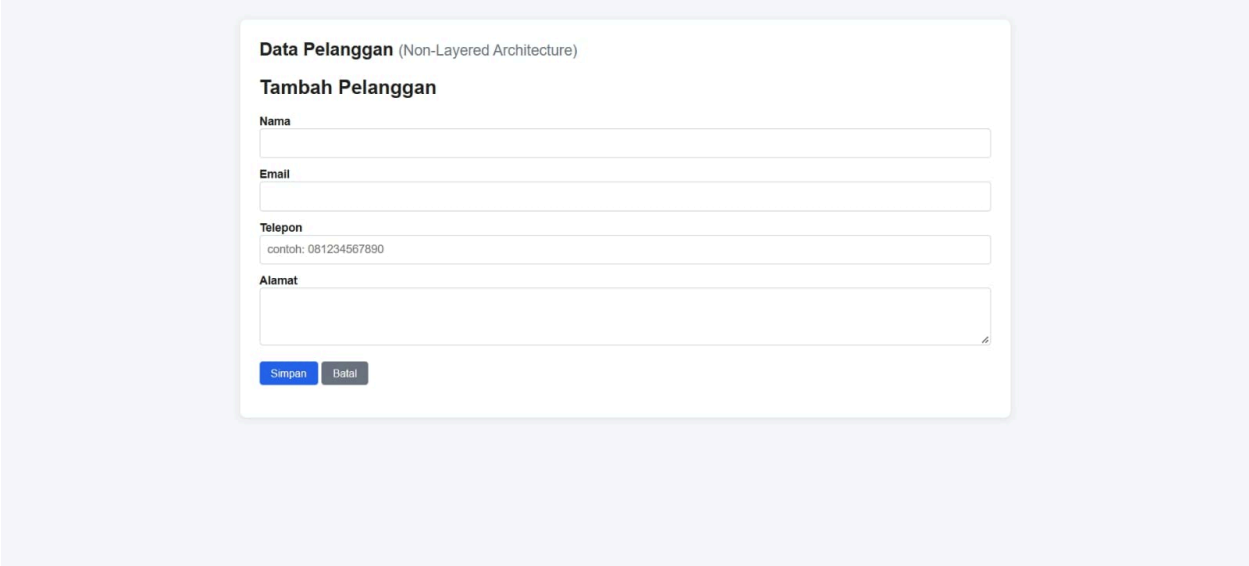
4.1 Pemenuhan Fitur

Tabel 4.1 Fitur CRUD

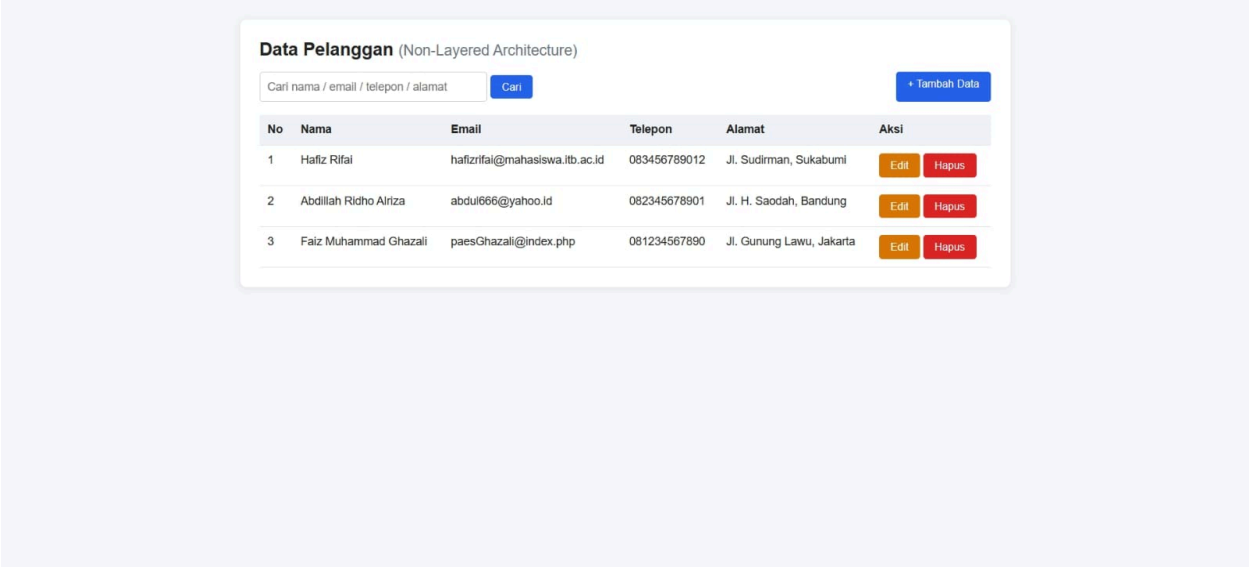
No	Fitur	Non-Layered	3-Layer
1	Tambah data	Ada (INSERT di index.php)	Ada (POST → Service → Repository)
2	Tampil data	Ada (SELECT, render tabel di PHP)	Ada (GET, tabel dibangun JavaScript)
3	Edit data	Ada (form edit, UPDATE)	Ada (PUT)
4	Hapus data	Ada (DELETE dengan konfirmasi)	Ada (DELETE dengan konfirmasi)
5	Pencarian	Ada (nama, email, telepon, alamat; sudah mencakup nama atau email)	Ada (nama atau email, sesuai ketentuan)
6	Validasi	Kolom wajib, format email, format telepon (8–15 digit); keunikan email lewat constraint UNIQUE dan penanganan error 1062	Kolom wajib, format email, keunikan email dicek lebih dulu di Service (via Repository) dan dijaga constraint UNIQUE; format telepon belum divalidasi

4.2 Tangkapan Layar

Non-Layered Website

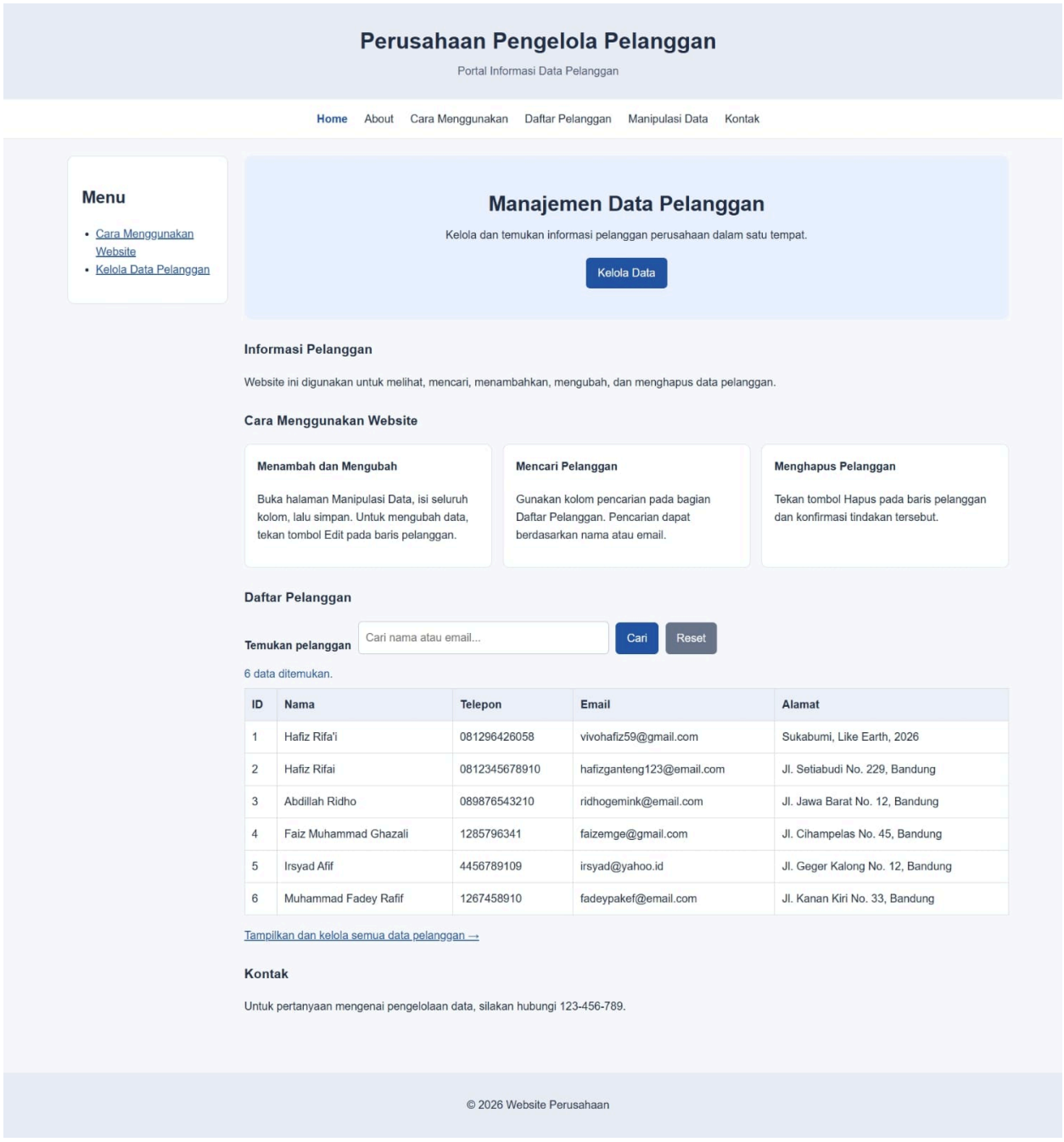


Gambar 4.1 Tampilan Form Input/Tambah Data Non-Layered

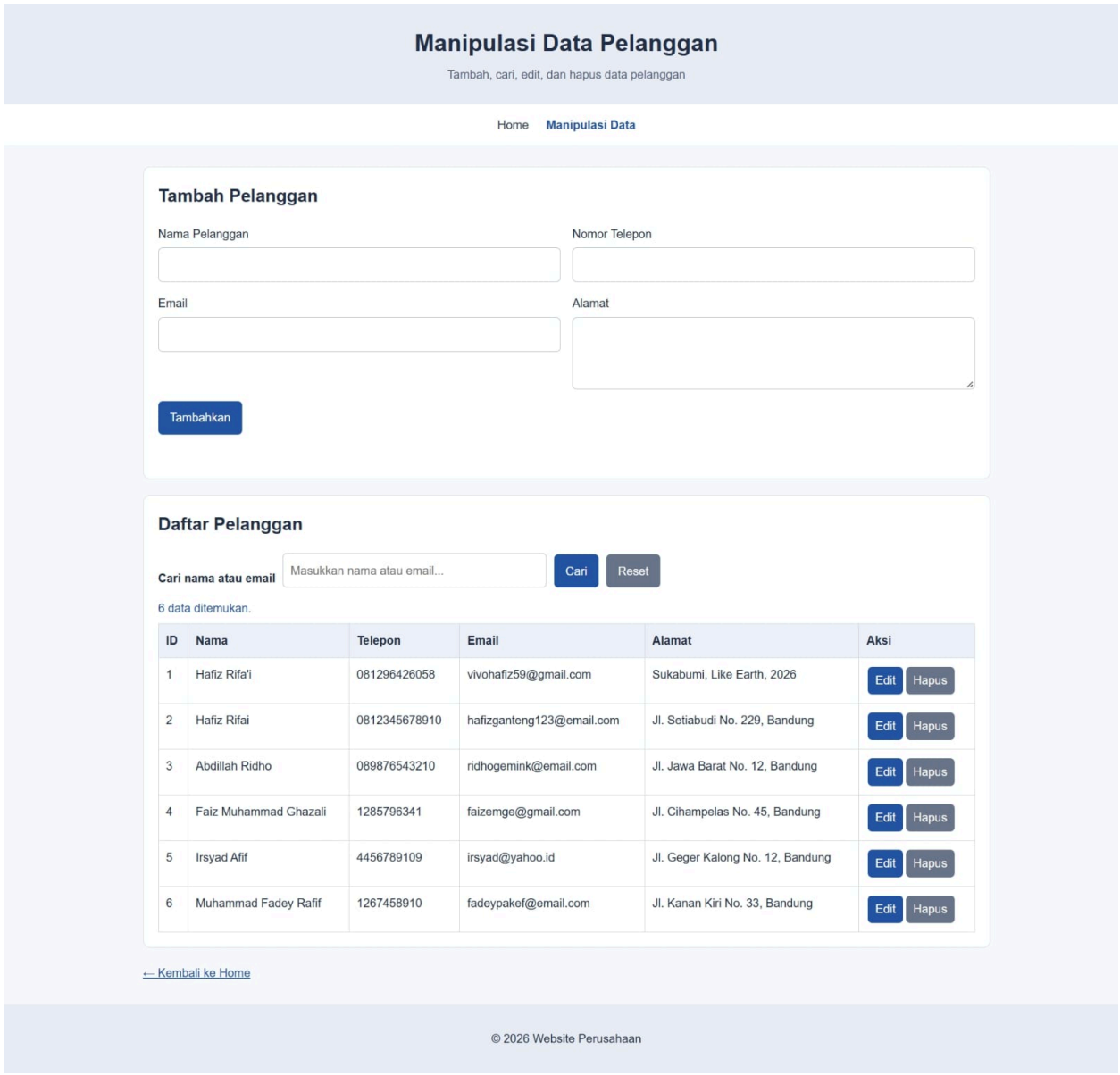


Gambar 4.2 Tampilan Tabel dan Daftar Data Non-Layered

3-Layer Website



Gambar 4.3 Tampilan Halaman Home 3-Layer



Gambar 4.4 Tampilan Halaman Manipulasi Data Pelanggan 3 -Layer

BAB V

ANALISIS PENGUJIAN DAN PERBANDINGAN

5.1 Perbandingan Kuantitatif Kode

Tabel 5.1 Perbandingan Kuantitatif Kode Non-Layered dan 3-Layer

Aspek	Non-Layered	3-Layer
Jumlah berkas kode utama	1 berkas PHP	4 berkas PHP backend + 2 HTML + 1 JS + 1 CSS
Baris kode (kira-kira)	±190 baris di satu berkas	±225 baris backend (api 54, service 68, repository 74, database 29) + ±300 baris frontend
Pemisahan HTML dan PHP	Menyatu	Terpisah (HTML statis dan JS, PHP hanya mengembalikan JSON)
Lokasi query SQL	Tersebar di `index.php`	Terpusat di `PelangganRepository`
Lokasi validasi	Di `index.php`, bercampur dengan tampilan	Terpusat di `PelangganService`
Format keluaran backend	HTML	JSON

5.2 Perbandingan Kualitatif

Tabel 5.2 Perbandingan Kualitatif Non-Layered dan 3-Layer

Aspek	Non-Layered	3-Layer
Struktur	Sederhana	Terpisah
FIle Program	Bisa satu file (index.php)	Beberapa komponen (terbagi per folder/layanan)

Aspek	Non-Layered	3-Layer
Frontend	Menyatu dengan PHP	Terpisah (HTML, CSS, JavaScript/Fetch API)
Business Logic	Menyatu dengan kode aplikasi (index.php)	Dipisahkan (PelangganService.php)
Database Access	Langsung dar PHP (mysqli)	Melalui komponen data access (PelangganRepository.php & Database.php)
Kemudahan Membuat Awal	Lebih mudah dan cepat; satu berkas, tanpa API	Lebih lama; perlu merancang endpoint, service, repository, dan JS
Keterbacaan	Baik pada aplikasi kecil, memburuk saat fitur bertambah	Lebih terstruktur; setiap berkas memiliki satu tanggung jawab
Pemeliharaan	Lebih sulit jika kompleks/perubahan berisiko	Lebih terstruktur & bug mudah dilokalisasi
Pengembangan	Cepat untuk aplikasi kecil, namun berkas membengkak jika fitur bertambah	Lebih terorganisasi; fitur baru ditambahkan per layer
Pengujian	Cenderung lebih sulit; sulit menguji logika tanpa UI	Lebih mudah dipisahkan; Service & Repository bisa diuji independen (misal via Postman)
Reusability	Rendah; logika terikat pada halaman HTML	Tinggi; API JSON dapat dipakai web lain atau aplikasi mobile
Kerja tim	Sulit dibagi karena berkas sama	Mudah dibagi per peran (frontend, logic, data)

BAB VI

KESIMPULAN DAN SARAN

8.1 Kesimpulan

1. Pemenuhan Fungsionalitas: Kedua pendekatan (Non-Layered dan 3-Layer Architecture) berhasil mengimplementasikan 6 fitur utama yang dipersyaratkan, yaitu tambah, tampil, edit, hapus, pencarian, dan validasi data pelanggan.

2. Kompromi Kecepatan vs Struktur (Trade-off): Non-Layered Architecture unggul dalam kecepatan pembuatan awal dan kesederhanaan deployment. Namun, penumpukan tampilan (HTML), logika bisnis, dan SQL dalam satu berkas menyebabkan keterbacaan kode memburuk (spaghetti code) serta menyulitkan proses pemeliharaan dan pengujian.

3. Kelebihan 3-Layer Architecture: Walaupun memerlukan struktur awal yang lebih kompleks, penerapan 3-Layer Architecture terbukti menghasilkan kode yang modular dan terisolasi. Hal ini memberikan kemudahan dalam isolasi bug, pengujian independen backend via Postman, fleksibilitas pembagian kerja tim, serta portabilitas API JSON yang dapat digunakan ulang (reusable) oleh platform lain seperti mobile apps.

4. Keamanan & Skalabilitas: 3-Layer Architecture memberikan batas keamanan yang lebih baik karena Data Layer terisolasi dari Presentation Layer (browser tidak berhubungan langsung dengan database).

8.2 Saran

1. Penyelarasan Aturan Validasi: Menyepadankan seluruh aturan validasi (seperti regex format nomor telepon 8–15 digit) pada PelangganService agar setara dengan Non-Layered.

2. Peningkatan Keamanan API: Mengimplementasikan mekanisme autentikasi (seperti JWT/Bearer Token), perlindungan CSRF, serta pembatasan Access-Control-Allow-Origin (CORS) pada endpoint api.php sebelum di-deploy ke lingkungan produksi.

3. Pengujian Otomatis (Automated Testing): Menerapkan pengujian unit (unit testing) menggunakan framework seperti PHPUnit khusus untuk PelangganService dan PelangganRepository.
4. Optimasi Antarmuka & Data: Menambahkan fitur paginasi (pagination) dan pembatas pencarian untuk menangani data pelanggan dalam jumlah besar agar tidak membebani memori browser.
5. Adopsi Framework Modern: Untuk proyek skala menengah-besar, disarankan bertransisi menggunakan framework backend (seperti Laravel) dan library frontend agar arsitektur 3-Layer terorganisasi secara standar otomatis.

Lampiran

- Lampiran A: Source code aplikasi Non-Layered (1-Layer/index.php, db_akademik.sql)

```
<?php
/*
 * APLIKASI CRUD PELANGGAN - NON-LAYERED ARCHITECTURE
 * Seluruh komponen berada dalam SATU file:
 *   1. Koneksi database
 *   2. Proses bisnis (validasi + query SQL)
 *   3. Tampilan (HTML)
  * Tidak ada pemisahan layer (presentation / business / data
access).
 */

// ===== 1. KONEKSI DATABASE (langsung di file aplikasi) =====
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
try {
    $conn = new mysqli("localhost", "root", "", "db_pelanggan");
    $conn->set_charset("utf8mb4");
} catch (mysqli_sql_exception $e) {
                    die("Koneksi     database     gagal:     "     .
htmlspecialchars($e->getMessage()));
}

$aksi   = $_GET['aksi'] ?? 'list';
$pesan  = $_GET['pesan'] ?? '';
$error  = '';
$data   = ['id' => '', 'nama' => '', 'email' => '', 'telepon' => '',
'alamat' => ''];

// ===== 2. PROSES BISNIS & QUERY (CREATE, UPDATE, DELETE) =====
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $act     = $_POST['aksi'] ?? '';
    id  = (int)(_POST['id'] ?? 0);
    nama  = trim(_POST['nama'] ?? '');
    email = trim(_POST['email'] ?? '');
    telepon = trim(_POST['telepon'] ?? '');
    alamat = trim(_POST['alamat'] ?? '');

    if ($act === 'hapus') {
```

```
          $stmt = $conn->prepare("DELETE FROM pelanggan WHERE id =
?");
        $stmt->bind_param("i", $id);
        $stmt->execute();
             header("Location: index.php?pesan=" . urlencode("Data
berhasil dihapus."));
        exit;
    }

    if ($act === 'tambah' || $act === 'ubah') {
        // Validasi (logika bisnis)
        $digitTelepon = preg_replace('/\D/', '', $telepon);
          if ($nama === '' || $email === '' || $telepon === '' ||
$alamat === '') {
            $error = "Semua field wajib diisi.";
        } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $error = "Format email tidak valid.";
            } elseif (!preg_match('/^\+?[0-9\s\-]+$/', telepon)||strlen(
digitTelepon) < 8 || strlen($digitTelepon) > 15) {
             $error = "Telepon tidak valid (8-15 digit, boleh diawali
+ dan memuat spasi atau tanda -).";
        }

        if ($error === '') {
            try {
                if ($act === 'tambah') {
                        $stmt = $conn->prepare("INSERT INTO pelanggan
(nama, email, telepon, alamat) VALUES (?,?,?,?)");
                            $stmt->bind_param("ssss", $nama, $email,
$telepon, $alamat);
                    $stmt->execute();
                                header("Location: index.php?pesan=" .
urlencode("Data berhasil ditambahkan."));
                } else {
                         $stmt = $conn->prepare("UPDATE pelanggan SET
nama=?, email=?, telepon=?, alamat=? WHERE id=?");
                           $stmt->bind_param("ssssi", $nama, $email,
$telepon, $alamat, $id);
                    $stmt->execute();
```

```
                                header("Location: index.php?pesan=" .
urlencode("Data berhasil diperbarui."));
                }
                exit;
            } catch (mysqli_sql_exception $e) {
                        error = (e->getCode() == 1062) ? "Email sudah
terdaftar." : "Kesalahan database: " . $e->getMessage();
            }
        }
        // Jika error, kembalikan input ke form
          $data = ['id' => $id, 'nama' => $nama, 'email' => $email,
'telepon' => $telepon, 'alamat' => $alamat];
        aksi = (act === 'tambah') ? 'tambah' : 'edit';
    }
}

// Ambil data untuk form edit (READ satu baris)
if ($aksi === 'edit' && $error === '') {
    id = (int)(_GET['id'] ?? 0);
    $stmt = $conn->prepare("SELECT * FROM pelanggan WHERE id = ?");
    $stmt->bind_param("i", $id);
    $stmt->execute();
    $row = $stmt->get_result()->fetch_assoc();
    if (!$row) {
         header("Location: index.php?pesan=" . urlencode("Data tidak
ditemukan."));
        exit;
    }
    $data = $row;
}

// Ambil daftar data (READ semua + pencarian)
cari = trim(_GET['cari'] ?? '');
if ($cari !== '') {
    like = "%cari%";
     $stmt = $conn->prepare("SELECT * FROM pelanggan WHERE nama LIKE
? OR email LIKE ? OR telepon LIKE ? OR alamat LIKE ? ORDER BY id
DESC");
    $stmt->bind_param("ssss", $like, $like, $like, $like);
```

```
    $stmt->execute();
    $hasil = $stmt->get_result();
} else {
      $hasil = $conn->query("SELECT * FROM pelanggan ORDER BY id
DESC");
}
?>
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>CRUD Pelanggan - Non-Layered</title>
<style>
    body { font-family: Arial, sans-serif; background:#f4f6f9;
margin:0; padding:24px; color:#222; }
      .box { max-width:900px; margin:auto; background:#fff;
padding:24px; border-radius:8px; box-shadow:0 2px 8px
rgba(0,0,0,.08); }
  h1 { margin-top:0; font-size:22px; }
  table { width:100%; border-collapse:collapse; margin-top:16px; }
    th, td { padding:10px; border-bottom:1px solid #e3e6ea;
text-align:left; font-size:14px; vertical-align:top; }
  th { background:#f0f3f7; }
    .btn { display:inline-block; padding:7px 14px; border:0;
border-radius:4px; cursor:pointer; text-decoration:none;
font-size:13px; color:#fff; background:#2563eb; }
  .btn.edit { background:#d97706; } .btn.hapus { background:#dc2626;
} .btn.abu { background:#6b7280; }
  .alert { padding:10px 14px; border-radius:4px; margin:12px 0; }
      .ok { background:#dcfce7; color:#166534; } .err {
background:#fee2e2; color:#991b1b; }
    label { display:block; margin-top:12px; font-weight:bold;
font-size:14px; }
    input[type=text], input[type=email], textarea { width:100%;
padding:9px; border:1px solid #cbd5e1; border-radius:4px;
box-sizing:border-box; font-family:inherit; font-size:14px; }
    .bar { display:flex; gap:8px; justify-content:space-between;
flex-wrap:wrap; }
  form.inline { display:inline; }
```

```
</style>
</head>
<body>
<div class="box">
                    <h1>Data           Pelanggan           <small
style="font-weight:normal;color:#6b7280">(Non-Layered
Architecture)</small></h1>

    <?php  if  (pesan):?><divclass="alertok"><?=htmlspecialchars(pesan)
?></div><?php endif; ?>
    <?php  if  (error):?><divclass="alerterr"><?=htmlspecialchars(error)
?></div><?php endif; ?>

  <?php if ($aksi === 'tambah' || $aksi === 'edit'): ?>
    <!-- ===== 3. TAMPILAN: FORM TAMBAH / EDIT ===== -->
       <h2><?=  $aksi  ===  'tambah'  ?  'Tambah  Pelanggan'  :  'Ubah
Pelanggan' ?></h2>
    <form method="post" action="index.php">
      <input type="hidden" name="aksi" value="<?= $aksi === 'tambah'
? 'tambah' : 'ubah' ?>">
        <input  type="hidden"  name="id"  value="<?=  (int)$data['id']
?>">

      <label>Nama</label>
                  <input   type="text"   name="nama"   value="<?=
htmlspecialchars($data['nama']) ?>" required>
      <label>Email</label>
                  <input   type="email"   name="email"   value="<?=
htmlspecialchars($data['email']) ?>" required>
      <label>Telepon</label>
                <input   type="text"   name="telepon"   value="<?=
htmlspecialchars($data['telepon'])     ?>"     placeholder="contoh:
081234567890" required>
      <label>Alamat</label>
                <textarea   name="alamat"   rows="3"   required><?=
htmlspecialchars($data['alamat']) ?></textarea>

    <p>
        <button type="submit" class="btn">Simpan</button>
        <a href="index.php" class="btn abu">Batal</a>
```

```
    </p>
    </form>

    <?php else: ?>
    <!-- ===== 3. TAMPILAN: DAFTAR DATA ===== -->
    <div class="bar">
        <form method="get" action="index.php">
           <input type="text" name="cari" placeholder="Cari nama /
email / telepon / alamat" value="<?= htmlspecialchars($cari) ?>"
style="width:280px;display:inline-block">
        <button class="btn" type="submit">Cari</button>
          <?php if ($cari !== ''): ?><a href="index.php" class="btn
abu">Reset</a><?php endif; ?>
        </form>
              <a href="index.php?aksi=tambah" class="btn">+ Tambah
Data</a>
    </div>

    <table>

<tr><th>No</th><th>Nama</th><th>Email</th><th>Telepon</th><th>Alamat
</th><th>Aksi</th></tr>
        <?php no = 1; while(r = $hasil->fetch_assoc()): ?>
        <tr>
            <td><?= $no++ ?></td>
            <td><?= htmlspecialchars($r['nama']) ?></td>
            <td><?= htmlspecialchars($r['email']) ?></td>
            <td><?= htmlspecialchars($r['telepon']) ?></td>
            <td><?= nl2br(htmlspecialchars($r['alamat'])) ?></td>
            <td style="white-space:nowrap">
                <a class="btn edit" href="index.php?aksi=edit&id=<?=
(int)$r['id'] ?>">Edit</a>
               <form class="inline" method="post" action="index.php"
onsubmit="return confirm('Hapus data ini?')">
                <input type="hidden" name="aksi" value="hapus">
                          <input type="hidden" name="id" value="<?=
(int)$r['id'] ?>">
                                        <button class="btn hapus"
type="submit">Hapus</button>
            </form>
```

```
            </td>
        </tr>
        <?php endwhile; ?>
                <?php  if  ($no  ===  1):  ?><tr><td  colspan="6"
style="text-align:center;color:#6b7280">Belum                    ada
data.</td></tr><?php endif; ?>
    </table>
    <?php endif; ?>
</div>
</body>
</html>
```

- Lampiran B: Source code aplikasi 3-Layer (folder tugas-3layer)

a. Frontend

```
<?php
/*
 * APLIKASI CRUD PELANGGAN - NON-LAYERED ARCHITECTURE
 * Seluruh komponen berada dalam SATU file:
 *   1. Koneksi database
 *   2. Proses bisnis (validasi + query SQL)
 *   3. Tampilan (HTML)
  * Tidak  ada  pemisahan  layer  (presentation  /  business  /  data
access).
 */

// ===== 1. KONEKSI DATABASE (langsung di file aplikasi) =====
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
try {
    $conn = new mysqli("localhost", "root", "", "db_pelanggan");
    $conn->set_charset("utf8mb4");
} catch (mysqli_sql_exception $e) {
                  die("Koneksi    database    gagal:    "    .
htmlspecialchars($e->getMessage()));
}

$aksi   = $_GET['aksi'] ?? 'list';
$pesan  = $_GET['pesan'] ?? '';
$error  = '';
```

```
$data   = ['id' => '', 'nama' => '', 'email' => '', 'telepon' =>
'', 'alamat' => ''];

// ===== 2. PROSES BISNIS & QUERY (CREATE, UPDATE, DELETE) =====
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $act     = $_POST['aksi'] ?? '';
    id   = (int)(_POST['id'] ?? 0);
    nama  = trim(_POST['nama'] ?? '');
    email = trim(_POST['email'] ?? '');
    telepon = trim(_POST['telepon'] ?? '');
    alamat = trim(_POST['alamat'] ?? '');

    if ($act === 'hapus') {
         $stmt = $conn->prepare("DELETE FROM pelanggan WHERE id =
?");
        $stmt->bind_param("i", $id);
        $stmt->execute();
           header("Location: index.php?pesan=" . urlencode("Data
berhasil dihapus."));
        exit;
    }

    if ($act === 'tambah' || $act === 'ubah') {
        // Validasi (logika bisnis)
        $digitTelepon = preg_replace('/\D/', '', $telepon);
         if ($nama === '' || $email === '' || $telepon === '' ||
$alamat === '') {
            $error = "Semua field wajib diisi.";
        } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $error = "Format email tidak valid.";
         } elseif (!preg_match('/^\+?[0-9\s\-]+$/', telepon)||strlen(
digitTelepon) < 8 || strlen($digitTelepon) > 15) {
                $error = "Telepon tidak valid (8-15 digit, boleh
diawali + dan memuat spasi atau tanda -).";
        }

        if ($error === '') {
            try {
                if ($act === 'tambah') {
```

```
                    $stmt = $conn->prepare("INSERT INTO pelanggan
(nama, email, telepon, alamat) VALUES (?,?,?,?)");
                        $stmt->bind_param("ssss", $nama, $email,
$telepon, $alamat);
                    $stmt->execute();
                            header("Location: index.php?pesan=" .
urlencode("Data berhasil ditambahkan."));
                } else {
                     $stmt = $conn->prepare("UPDATE pelanggan SET
nama=?, email=?, telepon=?, alamat=? WHERE id=?");
                        $stmt->bind_param("ssssi", $nama, $email,
$telepon, $alamat, $id);
                    $stmt->execute();
                            header("Location: index.php?pesan=" .
urlencode("Data berhasil diperbarui."));
                }
                exit;
            } catch (mysqli_sql_exception $e) {
                    error = (e->getCode() == 1062) ? "Email sudah
terdaftar." : "Kesalahan database: " . $e->getMessage();
            }
        }
        // Jika error, kembalikan input ke form
         $data = ['id' => $id, 'nama' => $nama, 'email' => $email,
'telepon' => $telepon, 'alamat' => $alamat];
        aksi = (act === 'tambah') ? 'tambah' : 'edit';
    }
}

// Ambil data untuk form edit (READ satu baris)
if ($aksi === 'edit' && $error === '') {
    id = (int)(_GET['id'] ?? 0);
      $stmt = $conn->prepare("SELECT * FROM pelanggan WHERE id =
?");
    $stmt->bind_param("i", $id);
    $stmt->execute();
    $row = $stmt->get_result()->fetch_assoc();
    if (!$row) {
            header("Location: index.php?pesan=" . urlencode("Data
tidak ditemukan."));
```

```
        exit;
    }
    $data = $row;
}

// Ambil daftar data (READ semua + pencarian)
cari = trim(_GET['cari'] ?? '');
if ($cari !== '') {
    like = "%cari%";
      $stmt = $conn->prepare("SELECT * FROM pelanggan WHERE nama
LIKE ? OR email LIKE ? OR telepon LIKE ? OR alamat LIKE ? ORDER
BY id DESC");
    $stmt->bind_param("ssss", $like, $like, $like, $like);
    $stmt->execute();
    $hasil = $stmt->get_result();
} else {
     $hasil = $conn->query("SELECT * FROM pelanggan ORDER BY id
DESC");
}
?>
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<meta         name="viewport"         content="width=device-width,
initial-scale=1">
<title>CRUD Pelanggan - Non-Layered</title>
<style>
   body  {  font-family:  Arial,  sans-serif;  background:#f4f6f9;
margin:0; padding:24px; color:#222; }
     .box   {   max-width:900px;   margin:auto;   background:#fff;
padding:24px;    border-radius:8px;    box-shadow:0    2px    8px
rgba(0,0,0,.08); }
  h1 { margin-top:0; font-size:22px; }
   table { width:100%; border-collapse:collapse; margin-top:16px;
}
    th,  td  {  padding:10px;  border-bottom:1px  solid  #e3e6ea;
text-align:left; font-size:14px; vertical-align:top; }
  th { background:#f0f3f7; }
```

```
    .btn { display:inline-block; padding:7px 14px; border:0;
border-radius:4px; cursor:pointer; text-decoration:none;
font-size:13px; color:#fff; background:#2563eb; }
       .btn.edit { background:#d97706; } .btn.hapus {
background:#dc2626; } .btn.abu { background:#6b7280; }
  .alert { padding:10px 14px; border-radius:4px; margin:12px 0; }
     .ok { background:#dcfce7; color:#166534; } .err {
background:#fee2e2; color:#991b1b; }
    label { display:block; margin-top:12px; font-weight:bold;
font-size:14px; }
   input[type=text], input[type=email], textarea { width:100%;
padding:9px; border:1px solid #cbd5e1; border-radius:4px;
box-sizing:border-box; font-family:inherit; font-size:14px; }
   .bar { display:flex; gap:8px; justify-content:space-between;
flex-wrap:wrap; }
  form.inline { display:inline; }
</style>
</head>
<body>
<div class="box">
                    <h1>Data          Pelanggan          <small
style="font-weight:normal;color:#6b7280">(Non-Layered
Architecture)</small></h1>

   <?php if (pesan):?>< divclass = "alertok" ><?= htmlspecialchars(pesan)
?></div><?php endif; ?>
   <?php if (error):?>< divclass = "alerterr" ><?= htmlspecialchars(error)
?></div><?php endif; ?>

  <?php if ($aksi === 'tambah' || $aksi === 'edit'): ?>
    <!-- ===== 3. TAMPILAN: FORM TAMBAH / EDIT ===== -->
      <h2><?= $aksi === 'tambah' ? 'Tambah Pelanggan' : 'Ubah
Pelanggan' ?></h2>
    <form method="post" action="index.php">
         <input type="hidden" name="aksi" value="<?= $aksi ===
'tambah' ? 'tambah' : 'ubah' ?>">
       <input type="hidden" name="id" value="<?= (int)$data['id']
?>">

      <label>Nama</label>
```

```
                  <input   type="text"   name="nama"   value="<?=
htmlspecialchars($data['nama']) ?>" required>
      <label>Email</label>
                  <input   type="email"   name="email"   value="<?=
htmlspecialchars($data['email']) ?>" required>
      <label>Telepon</label>
                <input   type="text"   name="telepon"   value="<?=
htmlspecialchars($data['telepon'])    ?>"    placeholder="contoh:
081234567890" required>
      <label>Alamat</label>
                <textarea   name="alamat"   rows="3"   required><?=
htmlspecialchars($data['alamat']) ?></textarea>

    <p>
        <button type="submit" class="btn">Simpan</button>
        <a href="index.php" class="btn abu">Batal</a>
    </p>
    </form>

    <?php else: ?>
    <!-- ===== 3. TAMPILAN: DAFTAR DATA ===== -->
    <div class="bar">
        <form method="get" action="index.php">
          <input type="text" name="cari" placeholder="Cari nama /
email / telepon / alamat" value="<?= htmlspecialchars($cari) ?>"
style="width:280px;display:inline-block">
        <button class="btn" type="submit">Cari</button>
         <?php if ($cari !== ''): ?><a href="index.php" class="btn
abu">Reset</a><?php endif; ?>
        </form>
            <a  href="index.php?aksi=tambah"  class="btn">+  Tambah
Data</a>
    </div>

    <table>

<tr><th>No</th><th>Nama</th><th>Email</th><th>Telepon</th><th>Ala
mat</th><th>Aksi</th></tr>
        <?php no = 1; while(r = $hasil->fetch_assoc()): ?>
        <tr>
```

```
            <td><?= $no++ ?></td>
            <td><?= htmlspecialchars($r['nama']) ?></td>
            <td><?= htmlspecialchars($r['email']) ?></td>
            <td><?= htmlspecialchars($r['telepon']) ?></td>
            <td><?= nl2br(htmlspecialchars($r['alamat'])) ?></td>
            <td style="white-space:nowrap">
              <a class="btn edit" href="index.php?aksi=edit&id=<?=
(int)$r['id'] ?>">Edit</a>
             <form class="inline" method="post" action="index.php"
onsubmit="return confirm('Hapus data ini?')">
                <input type="hidden" name="aksi" value="hapus">
                       <input type="hidden" name="id" value="<?=
(int)$r['id'] ?>">
                                        <button class="btn hapus"
type="submit">Hapus</button>
            </form>
            </td>
        </tr>
        <?php endwhile; ?>
                <?php if ($no === 1): ?><tr><td colspan="6"
style="text-align:center;color:#6b7280">Belum ada
data.</td></tr><?php endif; ?>
    </table>
    <?php endif; ?>
</div>
</body>
</html>
```

b. backend

api.php

```
<?php
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE,
OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(200);
```

```
    exit;
}

require_once 'config/Database.php';
require_once 'data/PelangganRepository.php';
require_once 'services/PelangganService.php';

$db = (new Database())->getConnection();
$repo = new PelangganRepository($db);
$service = new PelangganService($repo);

$method = $_SERVER['REQUEST_METHOD'];
$input  =  json_decode(file_get_contents("php://input"),  true)  ??
[];
$id = $_GET['id'] ?? ($input['id'] ?? null);

switch ($method) {
    case 'GET':
        $search = $_GET['search'] ?? '';
        $res = $service->getCustomers($search);
        http_response_code($res['code']);
        echo json_encode($res);
        break;

    case 'POST':
        $res = $service->createCustomer($input);
        http_response_code($res['code']);
        echo json_encode($res);
        break;

    case 'PUT':
        $res = $service->updateCustomer($id, $input);
        http_response_code($res['code']);
        echo json_encode($res);
        break;

    case 'DELETE':
        $res = $service->deleteCustomer($id);
        http_response_code($res['code']);
        echo json_encode($res);
```

```
        break;

    default:
        http_response_code(405);
            echo json_encode(["status" => "error", "message" =>
"Method tidak diizinkan."]);
        break;
}
```

PelangganRepository.php

```
<?php
class PelangganRepository {
    private $conn;

    public function __construct($db) {
        $this->conn = $db;
    }

        // Fitur 2 (Tampil pelanggan) & Fitur 5 (Pencarian
berdasarkan nama atau email)
    public function getAll($search = "") {
        if (!empty($search)) {
                $sql = "SELECT * FROM pelanggan WHERE nama LIKE
:search OR email LIKE :search ORDER BY id ASC";
            $stmt = $this->conn->prepare($sql);
            $stmt->execute(['search' => "%$search%"]);
        } else {
            $sql = "SELECT * FROM pelanggan ORDER BY id ASC";
            $stmt = $this->conn->query($sql);
        }
        return $stmt->fetchAll();
    }

    // Ambil 1 pelanggan berdasarkan ID
    public function findById($id) {
           $stmt = $this->conn->prepare("SELECT * FROM pelanggan
WHERE id = :id");
        $stmt->execute(['id' => $id]);
        return $stmt->fetch();
```

```
    }

    // Query pembantu untuk Fitur 6 (Memeriksa keunikan email)
    public function findByEmail($email, $excludeId = null) {
        $sql = "SELECT id FROM pelanggan WHERE email = :email";
        $params = ['email' => $email];

        if ($excludeId !== null) {
            $sql .= " AND id != :id";
            $params['id'] = $excludeId;
        }

        $stmt = $this->conn->prepare($sql);
        $stmt->execute($params);
        return $stmt->fetch();
    }

    // Fitur 1 (Tambah pelanggan)
    public function create($data) {
           $sql = "INSERT INTO pelanggan (nama, telepon, email,
alamat) VALUES (:nama, :telepon, :email, :alamat)";
        $stmt = $this->conn->prepare($sql);
        return $stmt->execute([
            'nama'    => $data['nama'],
            'telepon' => $data['telepon'],
            'email'   => $data['email'],
            'alamat'  => $data['alamat']
        ]);
    }

    // Fitur 3 (Edit pelanggan)
    public function update($id, $data) {
          $sql = "UPDATE pelanggan SET nama = :nama, telepon =
:telepon, email = :email, alamat = :alamat WHERE id = :id";
        $stmt = $this->conn->prepare($sql);
        return $stmt->execute([
            'id'      => $id,
            'nama'    => $data['nama'],
            'telepon' => $data['telepon'],
            'email'   => $data['email'],
```

```
            'alamat'  => $data['alamat']
        ]);
    }

    // Fitur 4 (Hapus pelanggan)
    public function delete($id) {
         $stmt = $this->conn->prepare("DELETE FROM pelanggan WHERE
id = :id");
        return $stmt->execute(['id' => $id]);
    }
}
```

PelangganService.php

```
<?php
class PelangganService {
    private $repo;

    public function __construct(PelangganRepository $repo) {
        $this->repo = $repo;
    }

    // Fitur 6: Validasi kelengkapan dan keunikan email
    private function validate($data, $excludeId = null) {
        $nama    = trim($data['nama'] ?? '');
        $telepon = trim($data['telepon'] ?? '');
        $email   = trim($data['email'] ?? '');
        $alamat  = trim($data['alamat'] ?? '');

          if (empty($nama) || empty($telepon) || empty($email) ||
empty($alamat)) {
             return "Seluruh data (nama, telepon, email, alamat)
wajib diisi.";
        }

        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            return "Format email tidak valid.";
        }
```

```
        if ($this->repo->findByEmail($email, $excludeId)) {
            return "Email sudah terdaftar. Gunakan email lain.";
        }

        return null; // Lolos validasi
    }

    public function getCustomers($search = "") {
        $data = $this->repo->getAll($search);
          return ["status" => "success", "code" => 200, "data" =>
$data];
    }

    public function createCustomer($data) {
        $error = $this->validate($data);
        if ($error) {
            return ["status" => "error", "code" => 422, "message"
=> $error];
        }

        $this->repo->create($data);
          return ["status" => "success", "code" => 201, "message"
=> "Pelanggan berhasil ditambahkan."];
    }

    public function updateCustomer($id, $data) {
        if (empty($id) || !$this->repo->findById($id)) {
            return ["status" => "error", "code" => 404, "message"
=> "Data pelanggan tidak ditemukan."];
        }

        $error = $this->validate($data, $id);
        if ($error) {
            return ["status" => "error", "code" => 422, "message"
=> $error];
        }

        $this->repo->update($id, $data);
          return ["status" => "success", "code" => 200, "message"
=> "Data pelanggan berhasil diperbarui."];
```

```
    }

    public function deleteCustomer($id) {
        if (empty($id) || !$this->repo->findById($id)) {
             return ["status" => "error", "code" => 404, "message"
=> "Data pelanggan tidak ditemukan."];
        }

        $this->repo->delete($id);
          return ["status" => "success", "code" => 200, "message"
=> "Pelanggan berhasil dihapus."];
    }
}
```

Database.php

```
<?php
class Database {
    private $host = "localhost";
    private $db_name = "db_customer_management";
    private $username = "root";
    private $password = ""; // Default XAMPP kosong
    public $conn;

    public function getConnection() {
        $this->conn = null;
        try {
            $this->conn = new PDO(
                    "mysql:host=" . $this->host . ";dbname=" .
$this->db_name,
                $this->username,
                $this->password
            );
                    $this->conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);

$this->conn->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
PDO::FETCH_ASSOC);
        } catch(PDOException $e) {
```

```
            http_response_code(500);
            echo json_encode([
                "status" => "error",
                      "message" => "Koneksi Database Gagal: " .
$e->getMessage()
            ]);
            exit;
        }
        return $this->conn;
    }
}
```

- Lampiran C: File database SQL

db_pelanggan.sql (Non-Layered)

```
-- Database untuk aplikasi Non-Layered (PHP Native + MySQL)
CREATE DATABASE IF NOT EXISTS db_pelanggan
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE db_pelanggan;

CREATE TABLE IF NOT EXISTS pelanggan (
  id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  nama VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  telepon VARCHAR(20) NOT NULL,
  alamat TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES
('Faiz Muhammad Ghazali', 'paesGhazali@index.php', '081234567890',
'Jl. Gunung Lawu, Jakarta'),
('Abdillah Ridho Alriza', 'abdul666@yahoo.id', '082345678901', 'Jl.
H. Saodah, Bandung'),
('Hafiz Rifai', 'hafizrifai@mahasiswa.itb.ac.id', '083456789012',
'Jl. Sudirman, Sukabumi');
```

databasedummy.sql (3-Layer)

```
CREATE DATABASE IF NOT EXISTS db_customer_management;
USE db_customer_management;

CREATE TABLE IF NOT EXISTS pelanggan (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nama VARCHAR(100) NOT NULL,
    telepon VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    alamat TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Data dummy awal untuk pengetesan
INSERT INTO pelanggan (nama, telepon, email, alamat) VALUES
('Hafiz Rifai', '0812345678910', 'hafizganteng123@email.com', 'Jl.
Setiabudi No. 229, Bandung'),
('Abdillah Ridho', '089876543210', 'ridhogemink@email.com', 'Jl.
Jawa Barat No. 12, Bandung'),
('Faiz Muhammad Ghazali', '1285796341', 'faizemge@gmail.com', 'Jl.
Cihampelas No. 45, Bandung'),
('Irsyad Afif', '4456789109', 'irsyad@yahoo.id', 'Jl. Geger Kalong
No. 12, Bandung'),
('Muhammad Fadey Rafif', '1267458910', 'fadeypakef@email.com', 'Jl.
Kanan Kiri No. 33, Bandung');
```